Eğitim - Doğrulama - Test setleri (Train - Validation - Test)

Makine öğrenmesinde veri seti genellikle üçe ayrılır:

- **Train (Eğitim) Seti**
  - Modelin öğrenmesi için kullanılır.
- **Validation (Doğrulama) Seti**
  - Hiperparametre ayarı ve model seçimi için kullanılır.
- **Test Seti**
  - Modelin gerçek performansını ölçmek için kullanılır.

Amaç: Modelin sadece veriyi ezberlemesini (overfitting) engelleyerek genelleme yeteneğini ölçmek.

Neden Veri Bölme Yapılır?

- Model tüm veriyi görürse: Gerçek hayatta başarısız olabilir (overfitting).
- Validation set: En iyi modeli seçmek için kullanılır.
- Test set:
  - En son aşamada, modelin gerçek başarımını ölçer.
  - Eğitim sürecine dahil edilmez.

```
from sklearn.model_selection import train_test_split
# %80 train, %20 test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Gradient Descent (Gradyan İnişi) Nedir?

Gradient Descent, bir makine öğrenimi modelinin hata oranını en aza indirmek için kullanılan optimizasyon algoritmasıdır.

Amaç: Modelin tahmin hatasını düşürmek için ağırlıkları güncelleyerek en düşük kayıp fonksiyonunu bulmak.
Nasıl çalışır?

- Ağırlıklar belirlenir.
- Kaybın (hata oranının) ne kadar büyük olduğu hesaplanır.
- Hata en aza inene kadar ağırlıklar küçük adımlarla güncellenir.
- Öğrenme oranı (α) güncelleme hızını belirler.

Adam (Adaptive Moment Estimation) Nedir?

Adam, derin öğrenme modellerinde yaygın olarak kullanılan, adaptif öğrenme oranına sahip bir optimizasyon algoritmasıdır.

Amaç: Her parametre için öğrenme oranını dinamik olarak ayarlayarak daha hızlı ve kararlı bir şekilde en düşük kayıp değerine ulaşmak.

- Parametreler bu adaptif oranlarla güncellenir.
- Hızlı yakınsama sağlar.
- Gürültülü ve seyrek verilerde iyi performans gösterir.
- Öğrenme oranını otomatik olarak ayarladığı için daha az manuel ayar gerektirir.
- Derin öğrenme modellerinde, büyük ve karmaşık veri setlerinde kullanılır.
- SGD'ye göre daha hızlı ve stabil sonuç verir.

Batch Gradient Descent (Veri Kümesi Gradyan İnişi)

- Tüm eğitim verisi kullanılır.
- Her iterasyonda tüm veri kümesiyle hesaplama yapılır.
- Kesin ama yavaş bir yöntemdir.
- Avantajları:
  - Daha doğru gradyan hesaplaması yapar.
  - Küçük veri kümelerinde iyi çalışır.
- Dezavantajları:
  - Büyük veri kümelerinde çok yavaş olur.
  - Bellek kullanımı yüksektir.

Stochastic Gradient Descent (SGD - Tekil Örnekli Gradyan İnişi)

- Her iterasyonda yalnızca bir veri örneğiyle hesaplama yapılır.
- Daha hızlıdır ama daha az kararlıdır.
- Hata yüzeyi üzerinde daha rastgele hareket eder.
- Avantajları:
  - Az bellek kullanır. Hızlıdır.
- Dezavantajları:
  - Gradyanlar düzensiz hareket eder, bazen yanlış yönde ilerleyebilir.
  - Daha fazla dalgalanma (oscillation) olabilir.

Mini-Batch Gradient Descent (Küçük Veri Kümesi Gradyan İnişi)

- Tüm veri yerine küçük gruplar (mini-batches) halinde çalışır.
- Batch ve SGD yöntemlerinin en iyi yönlerini birleştirir.
- Dengeyi sağlayan en yaygın kullanılan yöntemdir!
- Avantajları:
  - Bellek kullanımı batch yöntemine göre daha iyidir.
  - SGD'ye göre daha kararlı sonuçlar verir.
  - Paralel hesaplama ile hızlandırılabilir.
- Dezavantajları:
  - Batch kadar doğru olmayabilir.

Hangi Yöntemi Kullanmalıyız?

- Küçük veri setleri için → Batch Gradient Descent
- Gerçek zamanlı öğrenme için → SGD
- Büyük veri ve dengeli performans için → Mini-Batch Gradient Descent

Epoch

- Modelin tüm verilerle 1 kez eğitilmesine 1 epoch denilir.
- Örneğin veri sayısı 10.000, batch size 1000 epoch 5 ise
  - her 1000 veride 1 geri yayılım yapılır.
  - toplam 50 geri yayılım
- Epoch sayısı çok düşükse, model yetersiz öğrenir (underfitting).
- Epoch sayısı çok yüksekse, model ezberleyebilir (overfitting).
- Genellikle erken durdurma (early stopping) gibi yöntemlerle en uygun epoch sayısı bulunur.

Batch vs Mini-Batch vs SGD

```
# Batch Gradient Descent
model.fit(X, y, epochs=10, batch_size=len(X))

# Stochastic Gradient Descent (SGD)
model.fit(X, y, epochs=10, batch_size=1)

# Mini-Batch Gradient Descent
model.fit(X, y, epochs=10, batch_size=32)
```

Parametre - Hiperparametre

- **Parametreler** model tarafından eğitim esnasında öğrenilen değerlerdir.
- Bir yapay sinir ağında ağırlıklar (weights) ve biaslar parametrelerdir.
  - $y = w_1 x_1 + w_2 x_2 + b$
- **Hiperparametreler** model eğitilmeden önce belirlenen ayarlardır.
  - Eğitim sürecini ve modelin öğrenme şeklini etkiler.
  - Elle ayarlanır (deneme-yanılma veya optimizasyon teknikleri ile seçilir).
  - Öğrenme oranı
  - Epoch sayısı, Batch boyutu
  - Katman sayısı ve nöron sayısı
  - Aktivasyon fonksiyonu

Sınıflandırmada Aktivasyon ve Kayıp Fonksiyonları

Sigmoid Aktivasyon Fonksiyonu

- Sigmoid fonksiyonu, çıkışı 0 ile 1 arasında bir olasılık değeri olarak verir.
- $\sigma(x) = \frac{1}{1+e^{-x}}$
- Olasılık tahmini yapar.
- Büyük değerlerde gradyan sıfıra yaklaşabilir (vanishing gradient).

```
# Binary classification
model = models.Sequential([
    layers.Dense(1, activation='sigmoid')
])
```

Lineer Aktivasyon Fonksiyonu

- Lineer aktivasyon fonksiyonu, girdiyi doğrudan çıktı olarak verir.
- $f(x) = x$
- Genellikle regresyon problemlerinde kullanılır.
- Doğrusal olduğu için karmaşık (non-lineer) ilişkileri modelleyemez.

ReLU (Rectified Linear Unit) Aktivasyon Fonksiyonu

- Negatif değerleri sıfırlar, pozitif değerleri aynen geçirir.
- $f(x) = \max(0, x)$
- Hesaplama açısından çok verimlidir.
- Derin ağlarda vanishing gradient problemini azaltır.

```
# Binary classification
model = models.Sequential([
    layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='linear')  # regresyon
])
```

Softmax Aktivasyon Fonksiyonu

- Softmax, birden fazla sınıfın olasılıklarını hesaplar ve toplamı 1 olacak şekilde normalize eder.

$$S_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

```
# Multi-class classification
model = models.Sequential([
    layers.Dense(3, activation='softmax')
])
```

Binary Crossentropy (İkili Çapraz Entropi)

- İki sınıflı (binary) problemler için modelin hata oranını hesaplar.

$$L = -(y \log(p) + (1 - y) \log(1 - p))$$

Categorical Crossentropy (Kategorik Çapraz Entropi Kayıp Fonksiyonu)

- Birden fazla sınıf olan (multi-class) problemler için kullanılır.
- One-hot encoding ile çalışır.

$L = -\sum_i y_i \log(p_i)$

Regresyon Kayıp Fonksiyonları

Ortalama Kare Hata (Mean Squared Error, MSE)

- Gerçek değer ile tahmin arasındaki farkların karelerinin ortalamasını alır.
- $MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$
- Büyük hataları daha fazla cezalandırır (karesel büyüme)
- Türevi sürekli olduğu için optimizasyonu kolaydır
- **Ne zaman kullanılır?**
    - Outlier (aykırı değer) yoksa
    - Büyük hataların özellikle cezalandırılması isteniyorsa
    - Gaussian (normal) hata varsayımı varsa

Kayıp Fonksiyonları

```
# Binary classification
model.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)

# Multi-class classification
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy'
)

# Regression
model.compile(
    optimizer='adam',
    loss='mse'
)
```

Ortalama mutlak hata (Mean Absolute Error, MAE)

- Gerçek değer ile tahmin arasındaki mutlak farkların ortalamasını alır.
- $MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$
- Outlier'lara karşı daha dayanıklıdır
- Hataları lineer olarak cezalandırır
- Türevi sıfır noktasında tanımsızdır (optimizasyon biraz daha zor olabilir)
- **Ne zaman kullanılır?**
    - Veri setinde aykırı değerler varsa
    - Tüm hataların eşit önemde olduğu durumlarda
    - Daha robust (dayanıklı) model isteniyorsa

Ortalama Mutlak Yüzde Hata (Mean Absolute Percentage Error, MAPE)

- Hataları yüzde cinsinden ifade eder.
- $MAPE = \frac{100}{N} \sum_{i=1}^{N} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$
- Ölçekten bağımsızdır (relative hata verir)
- Yorumlaması kolaydır (% hata)
- $y_i = 0$ olduğunda tanımsızdır
- Küçük değerlerde hatayı aşırı büyütebilir
- **Ne zaman kullanılır?**
    - Sonuçların yüzde olarak yorumlanması isteniyorsa
    - Farklı ölçeklerdeki veriler karşılaştırılacaksa

MSE, MAE, MAPE Hesaplama

```python
import numpy as np

y_true = np.array([10, 20, 30, 40, 50])
y_pred = np.array([12, 18, 33, 37, 55])

mse = np.mean((y_true - y_pred)**2)
mae = np.mean(np.abs(y_true - y_pred))
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

print("MSE:", mse)
print("MAE:", mae)
print("MAPE:", mape)
```

Sınıflandırma Metrikleri ve Formülleri

TP, TN, FP, FN Kavramları

- True Positive (TP) - Gerçek Pozitif
  - Modelin "Pozitif" tahmin ettiği ve gerçekten pozitif olan örneklerdir.
- True Negative (TN) - Gerçek Negatif
  - Modelin "Negatif" tahmin ettiği ve gerçekten negatif olan örneklerdir.
- False Positive (FP) - Yanlış Pozitif
  - Modelin "Pozitif" tahmin ettiği ama aslında negatif olan örneklerdir.
- False Negative (FN) - Yanlış Negatif

  -Modelin "Negatif" tahmin ettiği ama aslında pozitif olan örneklerdir.

Bir sınıflandırma modelinin performansını değerlendirmek için kullanılan temel terimler:

Gerçek \ Tahmin	Pozitif (P)	Negatif (N)
Pozitif (P)	✅ TP (True Positive)	❌ FN (False Negative)
Negatif (N)	❌ FP (False Positive)	✅ TN (True Negative)

Örnek

Durum	TP	TN	FP	FN
Spam Filtresi	Spam e-postayı "Spam" olarak işaretlemek	Normal e-postayı "Normal" olarak bırakmak	Normal e-postayı "Spam" olarak yanlış işaretlemek	Spam e-postayı "Normal" olarak kaçırmak

Metrikler

Doğruluk (Accuracy)

Doğru tahmin edilen örneklerin, toplam örnek sayısına oranıdır.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Kesinlik (Precision)

Bir sınıfa ait olarak tahmin edilen örneklerin, gerçekten o sınıfa ait olma oranıdır.

$$Precision = \frac{TP}{TP + FP}$$

Duyarlılık (Recall)

Gerçekten pozitif olan örneklerin, model tarafından doğru tahmin edilme oranıdır.

$$Recall = \frac{TP}{TP + FN}$$

F1-Skoru

Kesinlik ve duyarlılığın harmonik ortalamasıdır. Dengesiz veri kümelerinde kullanışlıdır.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_true = [1,0,1,1,0,1,0,0,1,1]
y_pred = [1,0,1,0,0,1,1,0,1,0]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))
```

Karışıklık Matrisi (Confusion Matrix)

Her sınıfın gerçek ve tahmin edilen değerleri arasındaki ilişkileri gösteren matristir.

Gerçek \ Tahmin	**Pozitif (P)**	**Negatif (N)**
Pozitif (P)	TP (True Positive)	FN (False Negative)
Negatif (N)	FP (False Positive)	TN (True Negative)

Karışıklık Matrisi Örnek

```python
# Confusion Matrix (4 Sınıf) - Örnek Kod
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Gerçek sınıflar (0,1,2,3)
y_true = np.array([0,1,2,3,0,1,2,3,0,1,2,3])
# Tahminler
y_pred = np.array([0,2,1,3,0,1,2,2,0,0,2,3])

# Confusion Matrix hesaplama
cm = confusion_matrix(y_true, y_pred)
# Görselleştirme
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=[0,1,2,3])

disp.plot(cmap='Blues')
plt.title("4 Sınıflı Confusion Matrix")
plt.show()
```

Örnek Hesaplama

Eğer bir model:

- **Gerçek pozitif (TP) = 90**
- **Gerçek negatif (TN) = 50**
- **Yanlış pozitif (FP) = 30**
- **Yanlış negatif (FN) = 20**

için hesaplamalar:

- **Doğruluk (Accuracy)**:

$$Accuracy = \frac{90 + 50}{90 + 50 + 30 + 20} = 0.74$$

- **Kesinlik (Precision)**:

$$Precision = \frac{90}{90 + 30} = 0.75$$

- **Duyarlılık (Recall)**:

$$Recall = \frac{90}{90 + 20} = 0.818$$

- **F1-Skoru**:

$$F1 = 2 \times \frac{0.75 \times 0.818}{0.75 + 0.818} = 0.783$$